CO Attainment Automation System – Implementation Guide

1. Objective

The goal of this system is to automate Course Outcome (CO) attainment computation for academic assessments.

The automation pipeline already exists in Python (Colab). The next step is to convert this into a web-based

platform so faculty can upload files and automatically generate CO attainment reports.

The platform will use:

• React – Client Interface

• Express.js – Backend API

• Database – Store users, subjects, and files

• File storage – Store uploaded assessment files

2. High Level System Architecture

Client Layer:

React web application used by faculty and administrators.

Server Layer:

Express.js backend that exposes APIs for:

• Authentication

• File upload

• CO processing

• File storage and retrieval

• User management

Processing Layer:

Python services or scripts that execute the CO automation logic already implemented in Google Colab.

Storage Layer:

• Database for metadata (users, subjects, logs)

• File storage for uploaded Excel and question paper documents.

3. User Roles

Admin:

• Create new users

• Create subjects

• Assign faculty to subjects

• Grant or revoke admin privileges

• View all uploaded files

• Edit or replace files

• Download complete subject reports

• Send reminder emails to staff

Staff / Faculty:

• Login to system

• View assigned subjects

• Upload CAT / Assignment files

• Upload terminal marks

• Generate CO attainment reports

• Download generated reports

4. File Workflow

STEP 1 – Question Paper Upload

Faculty uploads CAT / Assignment question paper.

OCR + Regex logic extracts:

• Question numbers

• CO mapping

• Maximum marks

STEP 2 – Student Marks Upload

Faculty uploads student marks Excel exported from CAMU.

System maps:

• Student register number

• Question-wise marks

• Corresponding CO.

Output:

Question Paper Processed Excel file.

STEP 3 – Assessment Aggregation

At the end of the semester, the system uploads:

• CAT1

• CAT2

• Assignment1

• Assignment2

• Template file

System automatically fills the template and produces:

CO_ATTAINMENT_FINAL.xlsx

STEP 4 – Terminal Marks Processing

Faculty uploads:

• Terminal marks Excel file

• Previously generated CO attainment file.

User inputs:

• EP value

• Constraint value

• ELA values for each CO.

System calculates:

• Internal percentage

• Terminal contribution

• Final CO attainment

• Relative attainment.

5. Required Data Storage

Database tables:

Users

• id

• name

• email

• password hash

• role (admin / staff)

Subjects

• id

• subject_code

• subject_name

• assigned_staff

Files

• id

• subject_id

• file_type (CAT1, CAT2, ASS1, ASS2, QP, Terminal)

• file_path

• upload_date

Logs

• subject_id

• EP value

• constraint value

• ELA values

• generated report path

6. Admin Dashboard Features

Admin panel must allow:

• Create user accounts

• Assign subjects to staff

• Upload or replace files

• View subject progress

• Download subject files

• Monitor completion status

Admin should also see:

Visual analytics such as:

• CO attainment percentage charts

• Pass/fail distribution

• Internal vs terminal performance.

7. Staff Dashboard

Faculty should see only their assigned subjects.

Subject page should allow:

• Upload Question Paper

• Upload Student Marks

• Upload CAT/Assignment files

• Upload Terminal marks

• Enter EP / ELA values

• Generate final CO attainment report

• Download Excel outputs.

8. File Outputs to Maintain

Each subject folder should store:

- Question paper processed file
- Student marks integrated file
- CAT1 marks
- CAT2 marks
- Assignment 1 marks
- Assignment 2 marks
- Terminal marks
- Template filled file
- Final CO attainment Excel
- Logs of EP, ELA and constraint values.

9. Suggested Technology Stack

Frontend:

React + Material UI

Backend:

Node.js + Express.js

Database:

PostgreSQL or MongoDB Or Firebase Studio (required Free options)

File Storage:

Local server storage or cloud storage.

Processing Engine:
(https://colab.research.google.com/drive/1n45R4pOLwgvbouOuU8wNYvBUeLmgB8Tf?usp=sharing)

Python scripts (existing Colab code must be converted to backend service).

10. UI Design Goal

The interface should resemble a structured academic platform similar to Google Classroom.

Features:

- Clean dashboard
- Subject cards
- Upload panels
- Visual charts for attainment
- Download center for reports.